

3 - Déroulement détaillé des missions

3.1 Contexte, enjeux et choix technologiques

Avant d'écrire la moindre ligne de code, j'ai pris le temps de poser et de justifier la pile technologique du projet. Mon objectif n'a jamais été d'empiler des technologies à la mode, mais de répondre à des contraintes très concrètes de l'IDCS : du matériel mutualisé, des exigences de sécurité fortes, et le besoin de piloter beaucoup de machines à la fois sans s'effondrer.

Pourquoi le langage Go ? Pour toute la logique côté serveur, j'ai choisi Go. C'est un langage compilé, donc transformé en un programme natif rapide et léger, mais sa vraie force pour nous tient à sa gestion de la concurrence : grâce aux *goroutines*, un programme Go peut dialoguer avec des dizaines de machines en parallèle sans saturer la mémoire. Ce n'est pas un hasard si les grands outils d'infrastructure (Docker, Kubernetes, Terraform) sont écrits en Go ; c'est devenu le standard du domaine, et c'était le bon choix pour de l'orchestration de machines virtuelles.

Pourquoi Svelte pour l'interface ? Côté navigateur, j'ai retenu Svelte. Contrairement à des frameworks plus lourds comme React ou Angular, Svelte se comporte comme un compilateur : il transforme mon code en JavaScript pur, très léger, au moment de la compilation. Le résultat est une interface fluide et réactive, ce qui compte beaucoup quand on veut une expérience agréable aussi bien pour un professeur que pour un étudiant.



Le choix d'une architecture en microservices. L'application n'est pas un bloc unique. Je l'ai découpée en services : la partie qui gère les utilisateurs, les cours et les règles métier est séparée de celle qui parle techniquement à OpenStack pour créer les machines. Ce découpage est stratégique : si le service qui dialogue avec le

cloud rencontre un problème ou doit être redémarré, l'interface web, elle, continue de fonctionner. Cette isolation est ce qui rend l'ensemble robuste.

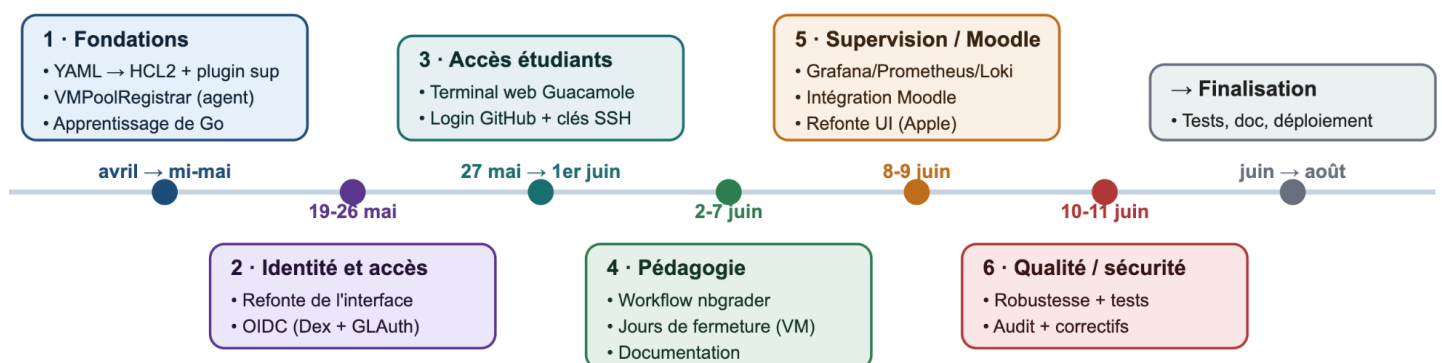
L'enjeu de la sécurité

La sécurité a été une préoccupation permanente, parce que l'application manipule des choses sensibles : des accès à de vraies machines, des clés de chiffrement (clés SSH) et des données personnelles comme les notes des étudiants. J'y reviens en détail dans la partie « Difficultés », mais l'idée à retenir ici est que j'ai mené une véritable démarche d'audit de mon propre code, puis appliqué des correctifs (authentification de toutes les routes, contrôle des rôles, protection contre les injections) pour garantir l'intégrité de l'infrastructure.

L'enjeu de la performance et de l'asynchronisme

Un professeur doit pouvoir demander 25 machines d'un coup. Or, créer ces machines sur le cloud prend plusieurs minutes. Si l'application attendait la fin du processus avant de répondre, le navigateur du professeur resterait figé. J'ai donc mis en place une **file de tâches asynchrones** : l'application enregistre la demande, rend immédiatement la main à l'utilisateur, et fabrique les machines en tâche de fond, tout en rafraîchissant l'interface en temps réel pour montrer l'avancement.

3.2 Phase 1 — Outils d'infrastructure et premières briques



Mon stage a commencé par une immersion dans les outils internes de l'IDCS, qui m'a permis de me familiariser avec Go sur des besoins concrets, avant de me lancer dans le projet principal.

3.2.1 Le problème du YAML et le plugin pour Sup

L'équipe utilise un outil réseau appelé `sup`, qui permet d'exécuter une même série de commandes simultanément sur des dizaines de machines. Historiquement, il se configure avec des fichiers au format YAML. Le souci, c'est que le YAML devient vite pénible à maintenir : il est très sensible aux erreurs d'indentation (un espace mal placé casse tout) et ne gère pas nativement la logique un peu avancée. C'était un point de friction quotidien pour l'équipe.

Pour y remédier, j'ai développé un convertisseur du YAML vers le **HCL2** (le langage de configuration de HashiCorp), pensé pour les infrastructures et bien plus lisible. J'ai ensuite écrit un **plugin** pour `sup`, en m'appuyant sur la technologie `go-plugin` de HashiCorp. Un plugin, c'est un adaptateur qu'on branche sur un programme existant pour lui ajouter une capacité sans toucher au programme d'origine. Grâce à ce travail, `sup` comprend désormais nativement le HCL2.



```
1  env:
2    IMAGE: "jupyterlab:tp-info"
3    PORT: "8888"
4
5  networks:
6    tp-salle-a:
7      hosts:
8        - "vmuser@10.0.0.11"
9        - "vmuser@10.0.0.12"
10
11  commands:
12    demarrer:
13      desc: "Demarrer JupyterLab"
14      run: "docker run -d -p $PORT:8888 $IMAGE"
```

```
1  variables {
2    image = "jupyterlab:tp-info"
3    port  = 8888
4  }
5
6  network "tp-salle-a" {
7    hosts = ["vmuser@10.0.0.11", "vmuser@10.0.0.12"]
8  }
9
10 command "demarrer" {
11   desc = "Démarrer JupyterLab"
12   run  = "docker run -d -p ${var.port}:8888 ${var.image}"
13 }
14
```

Yaml :

indentation piégeuse,
variables globales

HCL :

variables groupées,
liste compacte, blocs explicites,
réutilisation claire

3.2.2 VMPoolRegistrar : savoir quelles machines sont vivantes

Juste avant de m'attaquer à l'application principale, j'ai développé un second programme, séparé et lui aussi écrit en Go : **VMPoolRegistrar**. Le constat de départ était simple. Quand on crée des machines sur le cloud, rien ne dit automatiquement à la plateforme qu'elles existent, qu'elles ont fini de démarrer, ni si elles sont toujours en vie. L'inventaire devait être tenu à la main.

VMPoolRegistrar répond à ce problème par un mécanisme d'**auto-enregistrement**. Concrètement, c'est un petit agent qui est téléchargé sur chaque machine au moment de sa création, puis installé comme service permanent (il redémarre tout seul en cas de coupure). Dès que la machine est prête, l'agent « s'annonce » auprès du serveur central, puis envoie un **battement de cœur toutes les 15 secondes** pour signaler qu'il est toujours opérationnel. À l'extinction, il prévient proprement qu'il se retire.

Le serveur tient ainsi un **inventaire vivant** des machines, avec pour chacune un cycle de vie clair : *en démarrage* → *prête* → *en cours d'arrêt* → *morte*. Si une machine cesse d'envoyer son signal pendant plus d'une minute, elle est automatiquement repérée comme « périmée ».

Ce travail s'est révélé fondateur : le mécanisme d'auto-enregistrement conçu ici est devenu la colonne vertébrale du suivi en temps réel de VMPoolManager. C'est lui qui permet à l'interface du professeur d'afficher l'état exact de chaque machine, sans jamais avoir à rafraîchir la page.

3.2.3 Transition vers le projet principal

Cette première phase n'était pas un simple exercice. En développant ces outils, j'ai manipulé la syntaxe du langage en profondeur et compris les mécanismes de communication entre programmes. Surtout, ces briques (le convertisseur, l'agent d'enregistrement) ont été directement réutilisées pour configurer et suivre les machines de VMPoolManager, mon projet principal.

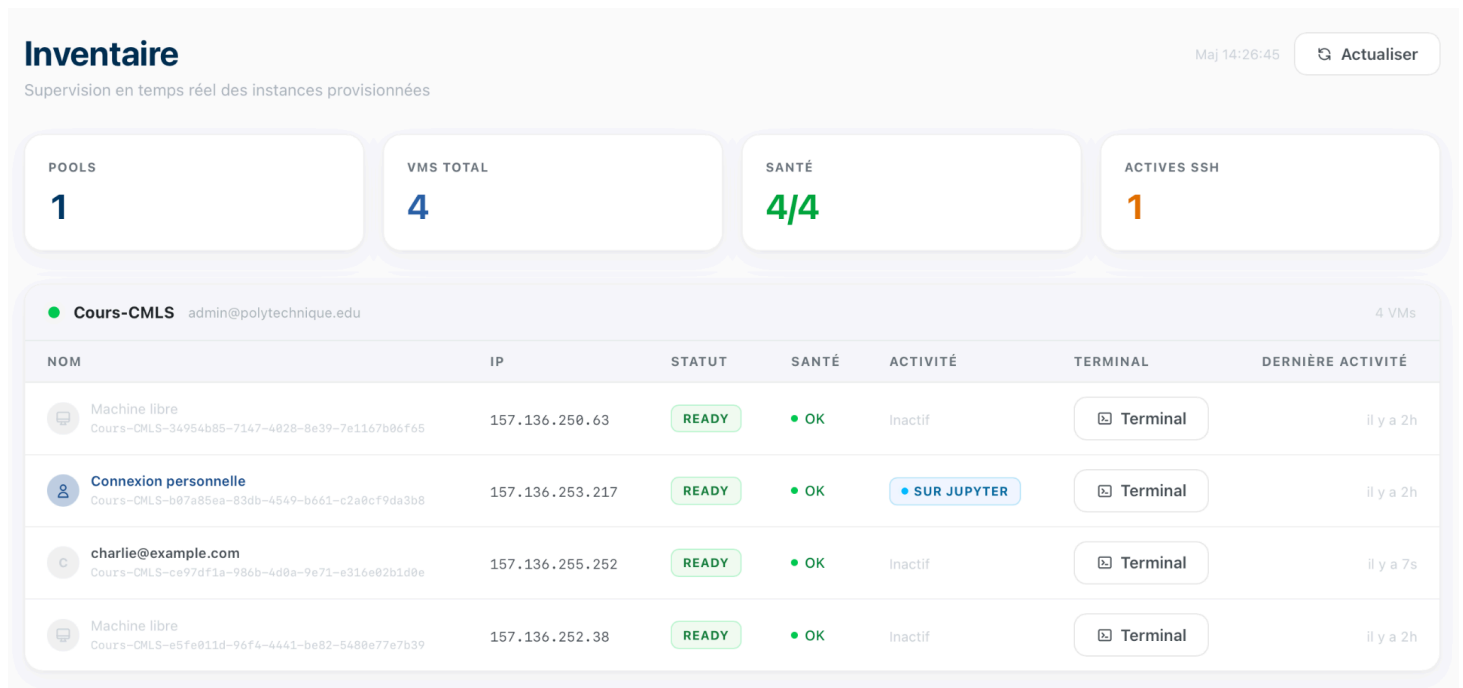


Figure : L'interface Regitrar implémentée à Vmpoolmanager

3.3 Phase 2 — Architecture et développement de VMPoolManager

3.3.1 Reprise de l'existant et choix de la refonte

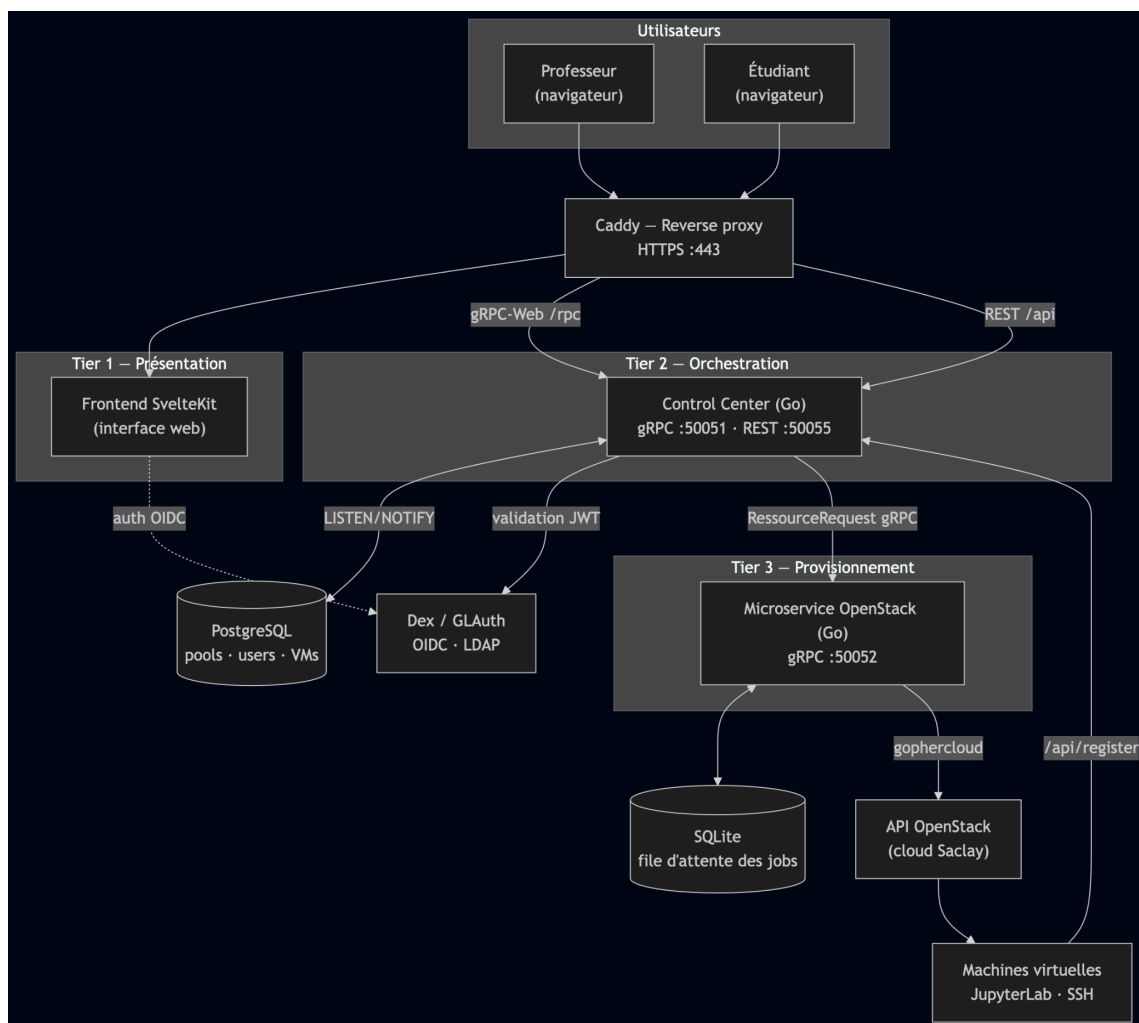
À l'origine, VMPoolManager existait sous forme de « preuve de concept » (PoC), initiée par un ancien alternant. Son travail avait le mérite de valider la faisabilité, mais le code souffrait d'une forte dette technique : aucune documentation, et une architecture monolithique qui rendait difficile l'ajout des fonctionnalités attendues (la correction des devoirs, le préchauffage des machines, etc.). Après une phase d'analyse, j'ai pris la décision — validée avec mon maître de stage — de repartir sur des bases saines et de reconstruire l'architecture.

3.3.2 Vue d'ensemble de l'architecture

La nouvelle architecture repose sur trois grands piliers :

- **L'interface (Front-End)** : ce que les utilisateurs voient et manipulent dans leur navigateur.
- **Le cerveau central (Control Center)** : le serveur qui applique les règles métier, gère les droits d'accès et conserve toutes les informations en base de données.
- **L'exécutant (microservice OpenStack)** : le programme dédié uniquement au dialogue technique avec le cloud.

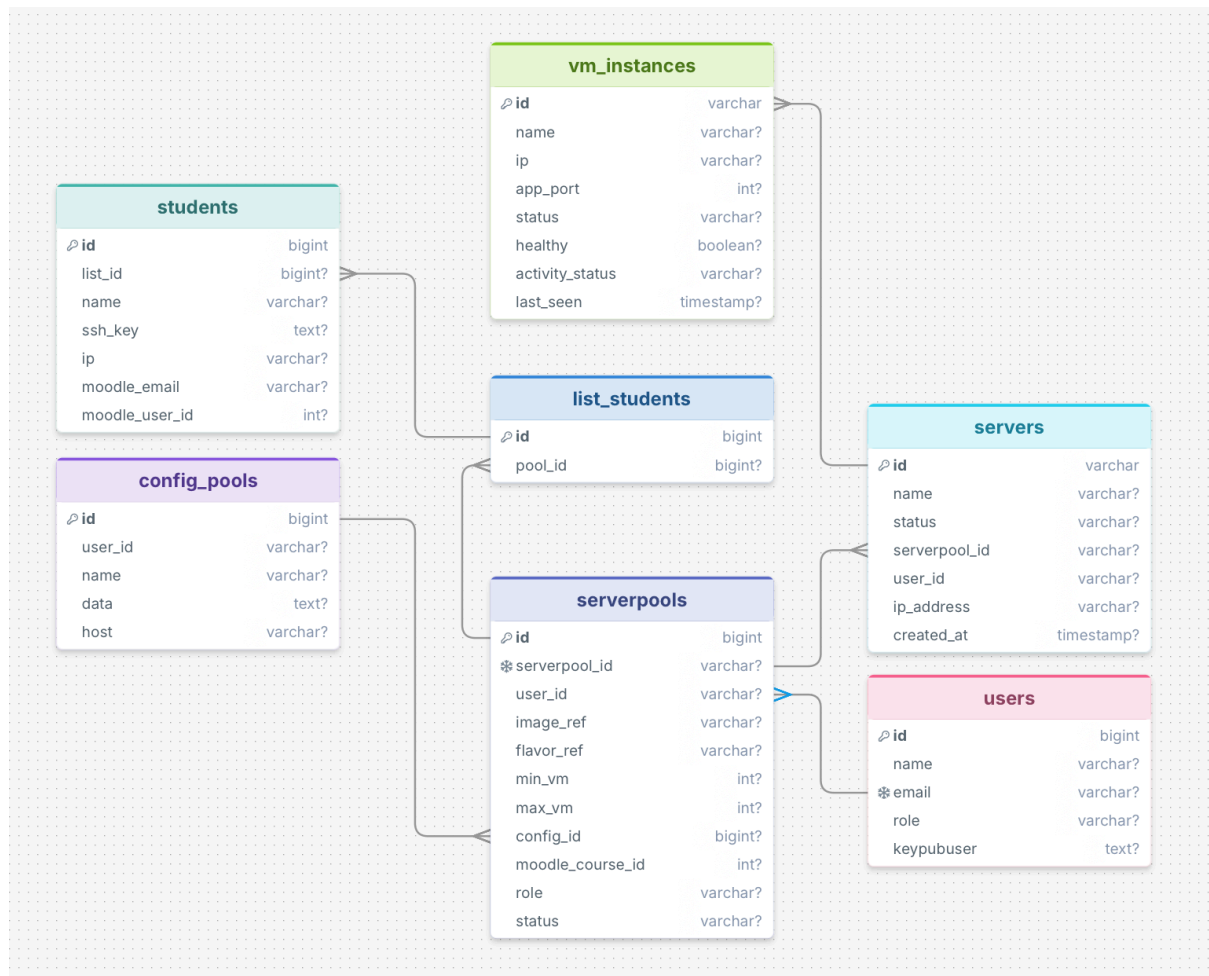
Un reverse proxy (Caddy) sert de porte d'entrée unique et sécurisée (HTTPS) vers l'ensemble.



3.3.3 Le modèle de données

Toutes les informations de la plateforme sont centralisées dans une base **PostgreSQL**. J'ai conçu ce modèle autour d'une idée directrice : **l'adresse e-mail**

comme **identité universelle**, qui relie un compte, un élève inscrit à un cours, et plus tard sa note. Les tables principales s'articulent ainsi : un professeur crée un ou plusieurs **pools** ; chaque pool possède une **liste d'élèves** (qui contient les **étudiants**), regroupe des **serveurs** (les machines virtuelles) et référence un **script d'installation** ; enfin, chaque serveur s'auto-enregistre dans la table d'**inventaire** alimentée par VMPoolRegistrar.



Ce choix de centralisation garantit la cohérence des données : supprimer un pool efface automatiquement, « en cascade », sa liste d'élèves, ce qui évite de laisser traîner des données orphelines.

3.3.4 Le back-end (Go + gRPC) et la file d'attente

Pour faire communiquer le cerveau central et l'exécutant, je n'ai pas utilisé une API web classique mais le protocole **gRPC**. Pour vulgariser, gRPC fonctionne comme une ligne téléphonique interne, rapide et typée, entre deux serveurs : un programme peut appeler une fonction d'un autre programme comme s'ils tournaient sur la même machine.

Pour gérer les fameuses créations en masse sans bloquer le système, j'ai mis en place une **file de tâches** (*job queue*) adossée à une petite base SQLite embarquée dans le microservice. Si OpenStack met du temps à répondre, la tâche attend sagement dans la file, sans jamais faire planter le reste de l'application. Des *workers* (ouvriers) piochent ensuite les tâches une à une.

3.3.5 Le front-end (Svelte) et le temps réel

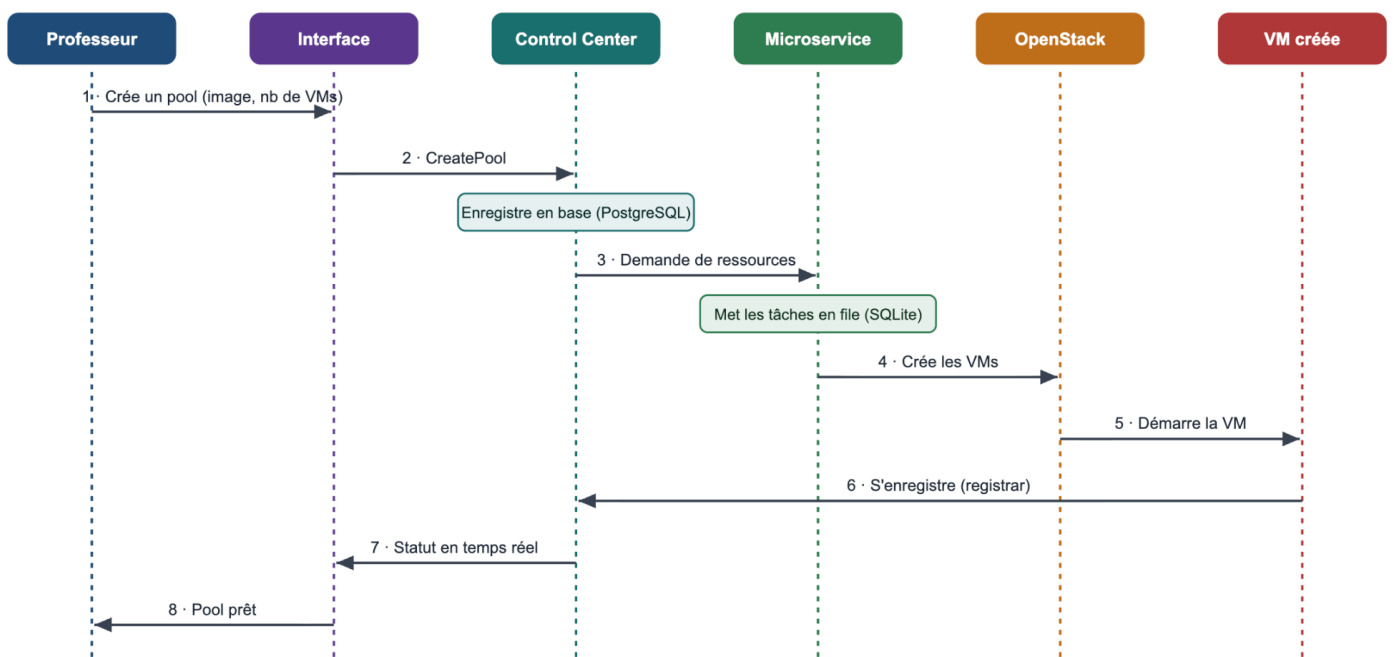
L'interface s'adapte automatiquement selon que l'on est professeur ou étudiant. L'un des défis les plus intéressants a été le « temps réel » : quand une machine finit de se créer, l'écran du professeur doit se mettre à jour tout seul. Pour cela, je me suis appuyé sur une fonctionnalité avancée de PostgreSQL, le mécanisme **LISTEN/NOTIFY** : dès qu'une donnée change en base, PostgreSQL pousse une notification au serveur Go, qui la relaie aussitôt au navigateur. L'interface se rafraîchit ainsi instantanément, sans que l'utilisateur n'ait rien à faire.

3.3.6 Intégration OpenStack

Le cœur du système, c'est le pilotage du cloud OpenStack via son API (Nova). L'application gère seule tout le cycle de vie des machines : création, listage, démarrage, extinction et suppression. J'y ai introduit la notion centrale de **pool** — un regroupement logique de machines associées à un cours. Pour respecter les **quotas** de l'université (la puissance et la mémoire allouées), j'ai aussi prévu la libération automatique des ressources : par exemple, les machines d'un cours sont **éteintes le week-end** plutôt que supprimées, puis rallumées, afin de ne pas gaspiller de ressources tout en conservant le travail des étudiants.

3.3.7 Création d'un pool et boucle de réconciliation

Côté professeur, le parcours est volontairement simple. Depuis son tableau de bord, il clique sur « Créer un pool », choisit l'image système (par exemple un environnement JupyterLab prêt pour Python), la puissance des machines et le nombre d'étudiants. En validant, tout se met en route en arrière-plan.



Mais le mécanisme le plus intéressant se cache derrière ce bouton. Plutôt que de créer les machines « une bonne fois » au moment du clic — ce qui serait fragile (que se passe-t-il si une machine plante en route ?) — j'ai adopté une approche de **réconciliation continue**, inspirée des grands orchestrateurs comme Kubernetes. Le principe : le professeur ne décrit pas des actions, mais un **état souhaité** (« je veux 25 machines »). Un programme de surveillance tourne en permanence et compare, **toutes les deux secondes**, l'état souhaité à l'état réel :

- s'il **manque** des machines, il met en file la création des manquantes ;
- s'il y en a **trop**, il programme la suppression des excédentaires ;
- en dehors des jours d'ouverture, il éteint les machines pour économiser les ressources.

Cette boucle rend le système **auto-réparant** : si une machine disparaît ou échoue à démarrer, le programme s'en aperçoit au cycle suivant et la recrée, sans intervention humaine. Un compteur de « tâches en cours » évite par ailleurs de relancer dix fois la même création pendant qu'OpenStack travaille.

Pour que l'étudiant n'attende pas plusieurs minutes, je maintiens en plus un petit **stock de machines déjà prêtes** (un pool « pré-chauffé ») : à la connexion, on lui en attribue une instantanément, et le stock se reremplit en tâche de fond.

3.3.8 Connexion et attribution (parcours étudiant)

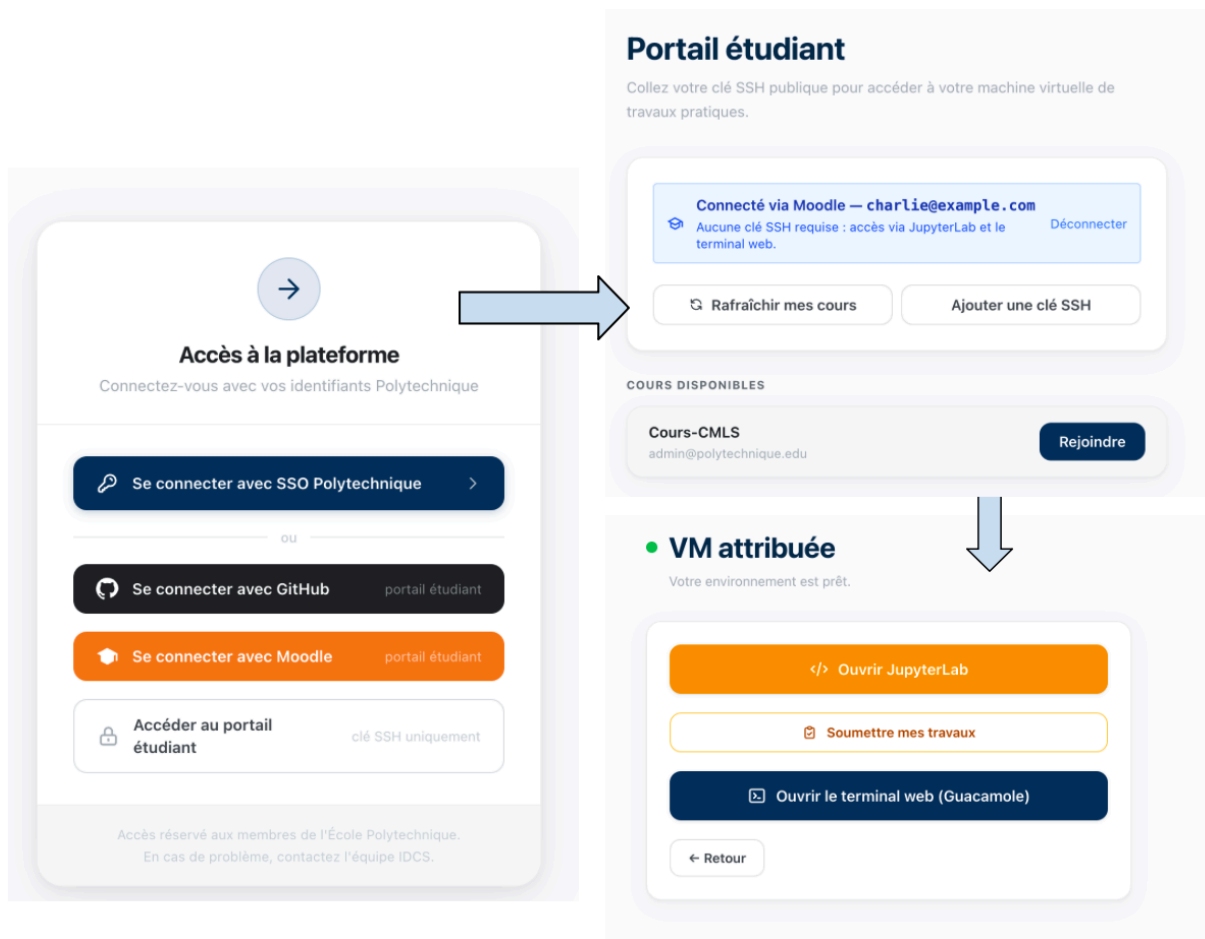


figure : workflow d'un étudiant pour aller se connecter sur sa VM

Côté étudiant, l'expérience se veut transparente. L'élève se connecte au portail avec ses identifiants Moodle ou son compte GitHub. L'application reconnaît à quel cours il appartient et lui attribue une machine disponible. S'il passe par GitHub, l'application récupère même automatiquement sa clé publique (clé SSH) et l'injecte dans la machine. En un clic, l'étudiant accède à son environnement de travail directement dans le navigateur — JupyterLab pour coder, ou un terminal distant via Guacamole — sans rien installer sur son ordinateur personnel.

3.3.9 Fonctionnalités supplémentaires : la correction des devoirs

Au-delà de la simple création de machines, j'ai développé un véritable flux pédagogique en intégrant l'outil **nbgrader** et l'API de **Moodle**. Le professeur peut distribuer un sujet sur toutes les machines de ses étudiants en un clic. L'application collecte les copies, lance la correction automatique, puis remonte les notes dans le carnet de notes Moodle de l'université. Le gain de temps pour l'enseignant est considérable.

Notation

Cours-CMLS (admin@polytechnique.edu) DevoirCMLS

ACTIONS

- Distribuer aux étudiants
- Collecter les copies
- Notation automatique**
- Exporter CSV

MOODLE

TP1 (démo) (/100)

Envoyer les notes vers Moodle

☐ Envoyer automatiquement après « Notation automatique »

[Carnet de notes Moodle ↗](#)

Espace de travail Cours-CMLS

Ouvrir JupyterLab Ouvrir Formgrader

JupyterLab et Formgrader s'ouvrent dans un onglet dédié.

VUE D'ENSEMBLE — DEVOIRCMLS

1 copies notées	19.0 moyenne	0 à corriger
--------------------	-----------------	-----------------

NOTES DES ÉTUDIANTS

charlie@example.com

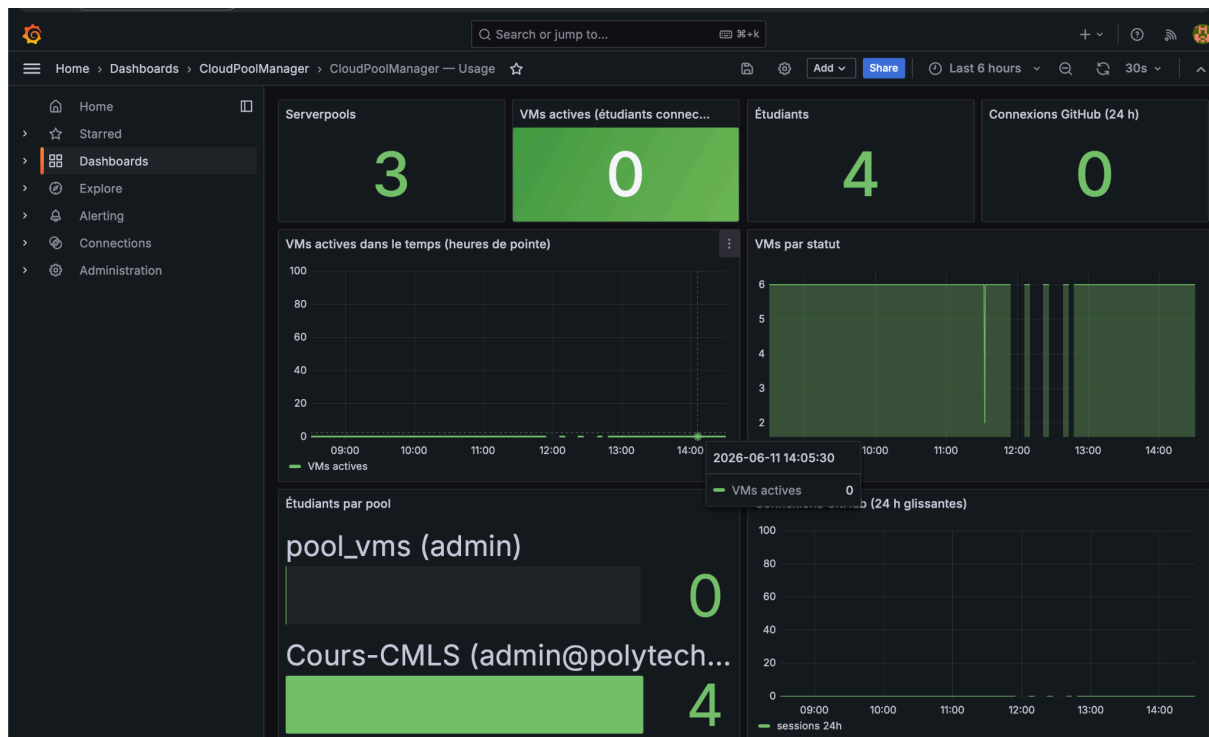
19.0/26.0 Corriger

1 notées 0 non rendu 19.0 moyenne

Précision importante. Au moment du déploiement réel à l'École, la remontée des notes se fera vraisemblablement **en lecture seule** dans un premier temps (consultation des données Moodle sans écriture des notes), le temps de valider les accès. L'écriture des notes restera donc désactivée en production au démarrage.

3.3.10 Observabilité de l'infrastructure

Enfin, pour garantir la stabilité du service, j'ai mis en place une pile d'observabilité complète : **Prometheus** pour collecter les métriques, **Loki** pour centraliser les journaux d'erreurs, et **Grafana** pour la visualisation. L'équipe de l'IDCS dispose ainsi de tableaux de bord en temps réel : elle suit d'un coup d'œil l'utilisation des quotas, le nombre de machines actives, et repère immédiatement la moindre anomalie.



Le but étant que lorsque l'application sera déployée, on pourra garder un oeil sur l'utilisation réel du service, afin de savoir si il y a un besoin d'augmenter les ressources comme la quantité de RAM ou de stockage afin de n'avoir aucun ralentissement, c'est aussi dans un but d'évolution du service.

4 — Difficultés rencontrées et solutions

Je tenais à consacrer une partie aux difficultés, parce qu'elles ont représenté une grande partie de l'apprentissage. En voici quatre, parmi les plus marquantes.

4.1 Des machines qui refusaient de se créer (images périmées)

À un moment, la création de machines a brutalement cessé de fonctionner. Après investigation, j'ai compris que l'identifiant d'une image système, stocké en base, ne correspondait plus à l'image réellement disponible côté OpenStack : l'image avait été reconstruite, et son identifiant avait changé. Le système gardait l'ancien et échouait silencieusement. J'ai résolu le problème en faisant **résoudre l'identifiant de l'image au dernier moment**, à partir d'une référence stable, plutôt que de me fier à une valeur figée qui pouvait devenir obsolète.

4.2 Le professeur et l'étudiant sur la même machine

J'ai constaté lors d'un test que je voyais apparaître, sur la machine d'un étudiant, un fichier que j'avais créé côté professeur. C'était un vrai problème : l'environnement de correction de l'enseignant (qui contient les corrigés et le carnet de notes) ne doit **jamais** être partagé avec un élève. La cause était une incohérence dans la façon dont la « machine de l'enseignant » était sélectionnée selon les endroits du code. J'ai corrigé cela en imposant **partout la même règle de sélection**, garantissant qu'un élève ne tombe jamais sur la machine de l'instructeur.

4.3 La distribution des copies qui plantait sur certains noms

La distribution d'un sujet « tournait dans le vide » pour un devoir précis. En lisant les journaux, j'ai vu que le nom du devoir contenait une **espace** (« Devoir corps »), et que ce nom était inséré tel quel dans des commandes système : le système le coupait en deux et la commande échouait. J'ai corrigé en **encadrant systématiquement** ces valeurs, puis, plus largement, en **validant en amont** tous les noms manipulés pour n'autoriser que des caractères sûrs. Cette correction m'a d'ailleurs mis sur la piste d'un sujet plus large : la sécurité.

4.4 L'audit de sécurité et le verrouillage des accès

En analysant mon propre code avec un regard d'attaquant, j'ai identifié deux familles de failles. D'une part, des **risques d'injection** : un nom de fichier ou de devoir bien choisi pouvait, en théorie, faire exécuter des commandes non prévues sur une machine. D'autre part, un problème d'**authentification** : certaines requêtes de l'interface n'étaient pas suffisamment contrôlées côté serveur, l'identité et le rôle (professeur ou étudiant) pouvant être déduits d'informations envoyées par le client plutôt que vérifiés.

J'ai appliqué des correctifs stricts : validation systématique des entrées pour fermer les injections, et surtout **authentification de toutes les routes** avec un contrôle de rôle côté serveur, de sorte qu'un étudiant ne puisse jamais déclencher une action réservée à un enseignant ni accéder aux données d'un autre. C'est l'une des contributions dont je suis le plus satisfait, parce qu'elle distingue un prototype « qui marche en démo » d'une application réellement digne de confiance.

[Figure : `fig-securite-auth` — le contrôle d'accès des requêtes]

Sources

- Documentation officielle Go — <https://go.dev/doc/>
- Documentation OpenStack — <https://docs.openstack.org/>
- Documentation Svelte / SvelteKit — <https://svelte.dev/docs>
- Documentation gRPC — <https://grpc.io/docs/>
- Documentation nbgrader — <https://nbgrader.readthedocs.io/>
- Documentation Moodle Web Services — <https://moodle.org/>
- [Ajouter les ressources internes IDCS consultées, avec l'accord de David Delavennat.]